Sensors Task Overview

Overview & Implementation Steps

Sensors Task Overview

- Sensors Task Overview
- Section Implementation Steps

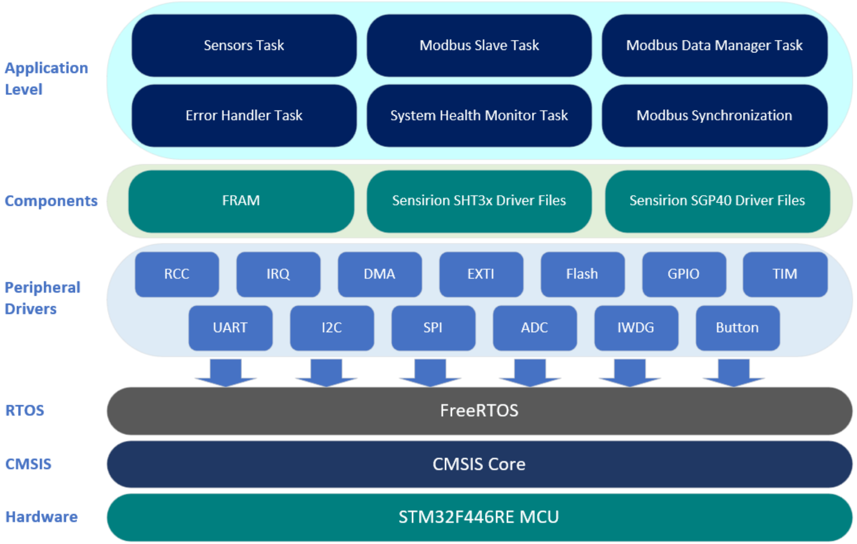
Sensors Task Overview Part I

General

- Purpose & Functionality: Acquires environmental data — temperature, humidity, and VOC index from Sensirion sensors over I2C. This data is then sent to Modbus Input Registers for user access.

- Project Significance:
    - **Environmental Sensing**: Forms the core functionality of the application, providing critical environmental data.

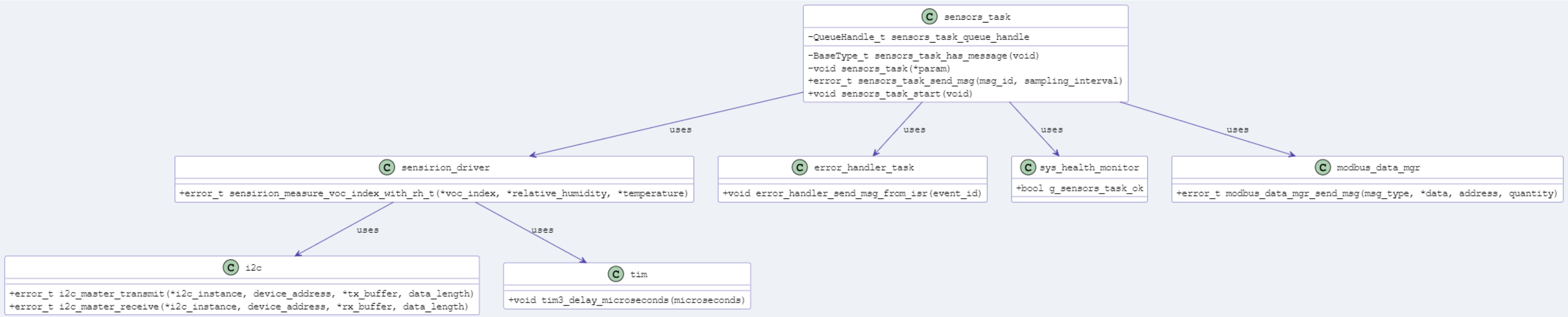
    - **User Interaction**: Enables users to access real-time environmental data via Modbus, facilitating informed decisions.



Sensors Task Overview Part II

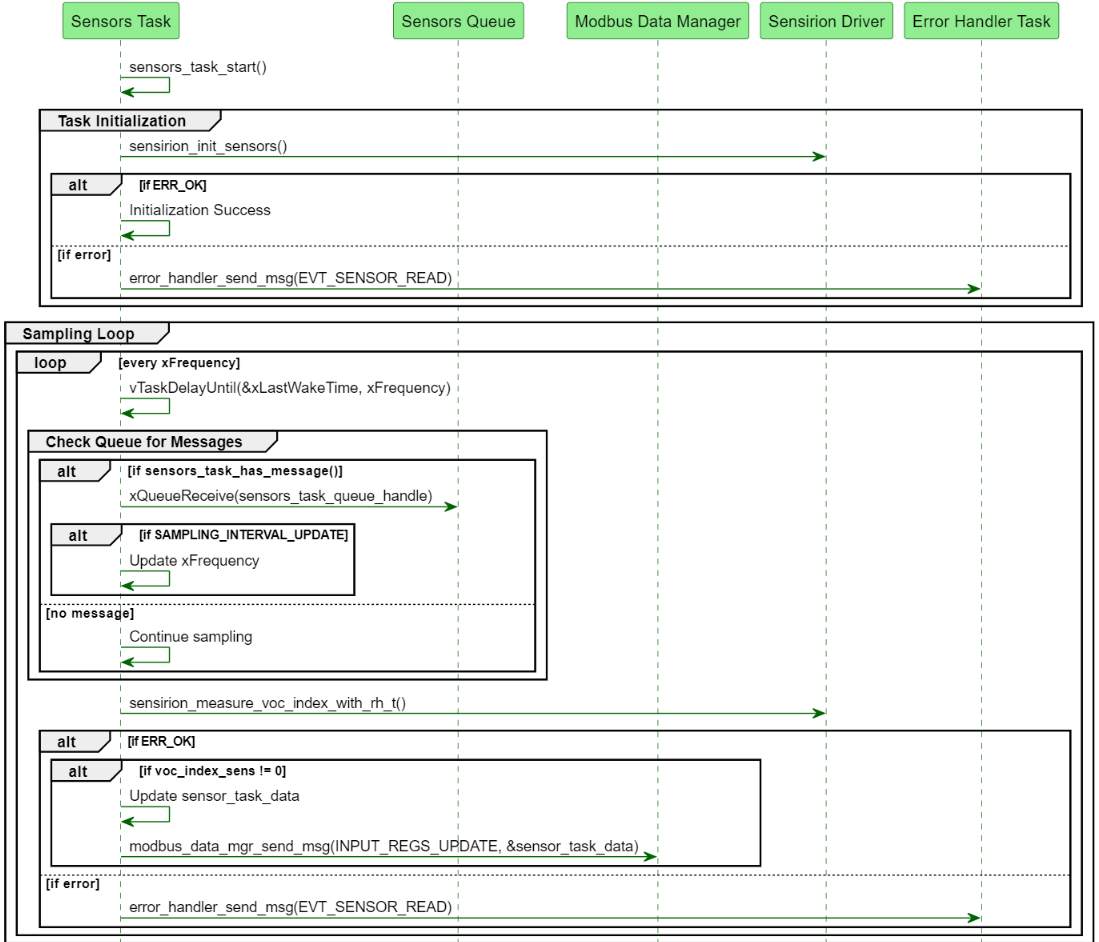
FreeRTOS Task Activities: Operates in a loop with a periodic delay, checks for incoming queue messages, collects sensor data, and communicates this data to the Modbus Data Manager. It also updates the g_sensors_task_ok system health flag and notifies the Error Handler of any issues.

- **Dependencies**: Depends on the Sensirion Driver (I2C & TIM) for sensor data acquisition, the Error Handler for reporting issues, the System Health Monitor for setting the global health flag, and the Modbus Data Manager for updating environmental sensor data (ambient temperature, humidity, and VOC index).



Sensors Task Overview Part III – Sequence Diagram

- **Task Initialization**: Initializes the sensors and checks for errors.
- **Sampling Loop**: Runs at a regular interval (defined by xFrequency) to gather sensor data.
- **Queue Message Handling**: Checks for and processes any messages in the queue, such as updates to the sampling interval.
- **Data Acquisition**: Collects data from the sensors and updates the Modbus Data Manager with the environmental sensor data.
- **Error Handling**: Notifies the Error Handler Task if there are issues during data acquisition.



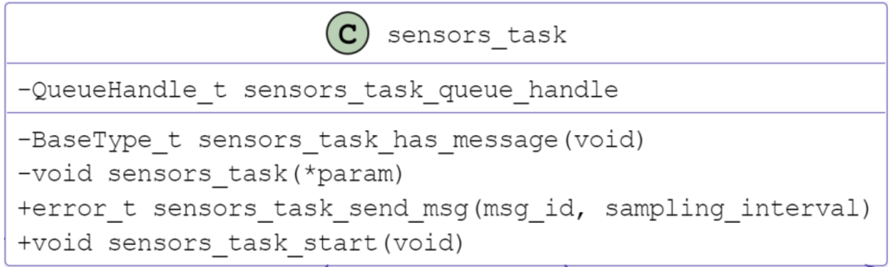
Sensors Task Files Preparation

Overview

- Objective: Prepare template .c/.h files for integrating/developing additional components, e.g., I2C, Sensirion Drivers, and TIM.

- Key Steps:
  - Create sensors_task.h with sensor data structure for VOC, ambient temperature and humidity data, sensors task message queue structure for updating the sampling interval (later over Modbus), and function prototypes.
  - Create sensors_task.c with function for checking for queue messages, the sensors task itself, and public functions (task start and send message).

- Project Significance: Sets up the Sensors Task for data acquisition and connection to Modbus software components.



I2C Driver Overview & Integration

Overview

- Objective: Provide an overview and demonstrate the integration of a custom interrupt-driven I2C driver using semaphore synchronization to efficiently handle communication within an RTOS environment.

- Key Steps:
  - **Understand**: The structure and design of the custom I2C driver.
  - **Integrate and Review**: Header file definitions, structures, static inline functions and public function prototypes.
  - **Integrate and Review**: Source file functions and interrupt handlers.

- Project Significance:
  - Gain an understanding of how the I2C driver synchronizes tasks with interrupts using semaphores, ensuring efficient communication in a real-time system.

Sensirion Drivers Overview & Integration

Overview

- Objective: Provide an overview of the Sensirion Drivers for the Temperature & Humidity (SHT3x) and VOC Index (SGP40) sensors and integrate them into the project.

- Key Steps:
  - **Overview**: Review the Sensirion Driver resources for both the temperature and humidity sensor (SHT3x) and VOC Index sensor (SGP40), contents and steps for integration.
  - **Integration**: Include the relevant files into the project and update for project/source code compatibility.

- Project Significance: Prepares the source code for communication with the temperature & humidity sensor and VOC Index sensor for use by the Sensors Task.

TIM3 Creation & Sensirion Driver Update

Overview

- Objective: Introduce and integrate the TIM3 microsecond delay function for use with the Sensirion driver, implementing semaphore synchronization to handle delay completion via the TIM3 interrupt handler.

- Key Steps:
    - **Create the tim3_delay_microseconds() function**: Utilize the TIM3 timer peripheral to generate a precise delay in microseconds.
    - **Update the Sensirion Driver Code**: Integrate this function using semaphore synchronization with the timer interrupt.
- Project Significance:
    - Finalizes the Sensirion driver implementation, allowing the system to handle sensor communication with accurate timing and demonstrates how to create a microsecond timer delay using TIM3 with interrupt and semaphore synchronization.

Sensors Task Update and Testing

Overview

- Objective: Integrate Sensirion Driver functions into the Sensors Task and perform basic sensor functionality testing.

- Key Steps:
    - **Integration**: Call `sensirion_init_sensors` at task initialization & `sensirion_measure_voc_index_with_rh_t` within the Sensors Task's main loop.
    - **Testing**: Start the sensors task, run the project, and verify successful sensor data acquisition and message reception.

- Project Significance: Confirms that the Sensors Task is operational, and that the application is ready for further development by integrating Modbus communication.

Next: Sensors Task Files Preparation